Práctica 2. Configuración de proyecto para RM.

1- Introducción

El objetivo de la práctica crear una aplicación de realidad mixta sencilla donde se pueda mostrar un objeto sobre una mesa del entorno. El objeto que vamos a utilizar es la primitiva Suzanne de Blender, que se suministra en la tarea del proyecto como fichero en formato FBX.

Después de completar todos los apartados de la práctica, conseguiremos que el objeto se active al pulsar el botón A del controlador izquierdo y se desactive al volverlo a pulsar. Cuando el objeto está activo, se ilumina y gira lentamente. Cuando está desactivado, permanece estático y sin iluminación.

2- Configuración del prefab del objeto

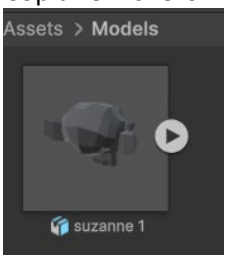
Vamos a generar un prefab para luego insertar en la posición adecuada haciendo uso de las librerías de Meta. Para poder desarrollar la práctica, el prefab debe cumplir estas condiciones:

- **Tamaño real.** Debe tener el tamaño real con el que queremos que se muestre en el entorno. En este caso, vamos a forzar para que la altura del modelo sea exactamente de 30 cm.
- **Pivote en la base.** El origen del prefab está en el centro de la base de la pieza, que es lo que se apoya en la superficie.
- **Cara frontal mirando hacia +Z.** Unity toma +Z como la dirección a la que mira el objeto, así que nos interesa que la cara principal del objeto esté orientada en esa dirección.
- **Box Collider.** Para poder detectar colisiones con otros objetos del entorno.
- **Iluminación.** El prefab debe incluir una spot light para poder iluminar el objeto posteriormente si lo consideramos necesario.

Para conseguir esto, seguimos los siguientes pasos:

a) Importar el objeto al proyecto

- Copiar el fichero FBX en la carpeta Models dentro de los Assets del proyecto.

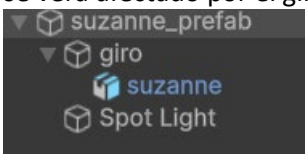


b) Creación de la jerarquía de objetos para el prefab

Va a haber un objeto padre que agrupe todos los objetos necesarios.

- Crear un Empty de nombre *suzanne_prefab*. Se trata del objeto padre que va a agrupar todos los objetos hijos necesarios. Situado en (0,0,0) y con escala (1,1,1)
- Crear un Empty hijo de *suzanne_prefab* de nombre *giro*. Se encargará de aplicar una velocidad de rotación de forma independiente a los objetos que cuelguen de él. Situado en (0,0,0) y con escala (1,1,1).

- Arrastrar el fichero FBX sobre el Empty *giro*. Se crea de esta forma un GameObject hijo *suzanne* con el modelo Suzanne. Dejarlo situado inicialmente en (0,0,0) y con escala (1,1,1). Habrá que ajustar su tamaño, posición y rotación para que sean adecuados.
- Crear un Spot Light hijo de *suzanne_prefab* de nombre *Spot Light*. Este GameObject no se verá afectado por el giro (no tiene sentido que la luz gire también).



c) Ajustes de tamaño, posición y rotación del modelo

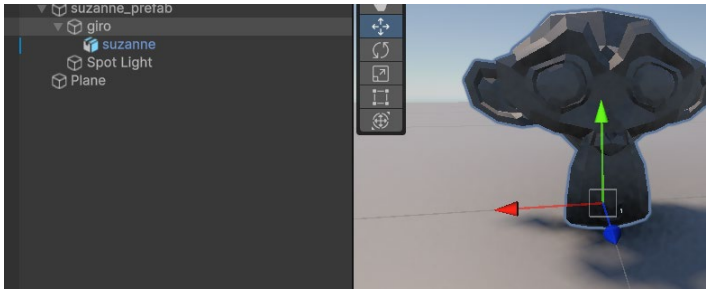
En primer lugar, vamos a ajustar el tamaño del modelo (GameObject *suzanne*). Para que mida 30 cm de alto, insertamos un cubo en la escena (por defecto, mide 1 m de lado) y le ajustamos la escala para que sea exactamente de 30 cm de alto. Con esa medida de referencia, escalamos el modelo.

A continuación, desplazamos el modelo (GameObject *suzanne*) hacia arriba para que la parte baja del modelo quede a altura 0. Podemos ayudarnos de un plano a altura 0 para tener referencia.

Los objetos utilizados como referencia (cubo y plano) se deben eliminar una vez ajustadas las propiedades del modelo.

Comprobamos a continuación que el eje local Z del GameObject *giro* sale perpendicular a la cara principal del objeto. En el caso del FBX que se suministra, no sale perpendicular (Suzanne mira en otra dirección), por lo que es necesario aplicar una rotación al objeto hijo *Suzanne* para que la cara esté mirando en la dirección correcta.

NOTA: Esto se podría corregir modificando el modelo en una aplicación de modelado externa como Blender. Pero es posible corregirlo sin necesidad de salir de Unity con el procedimiento indicado (aplicar rotación al objeto hijo *suzanne*, y dejar el objeto *giro* igual).



d) Creación de material

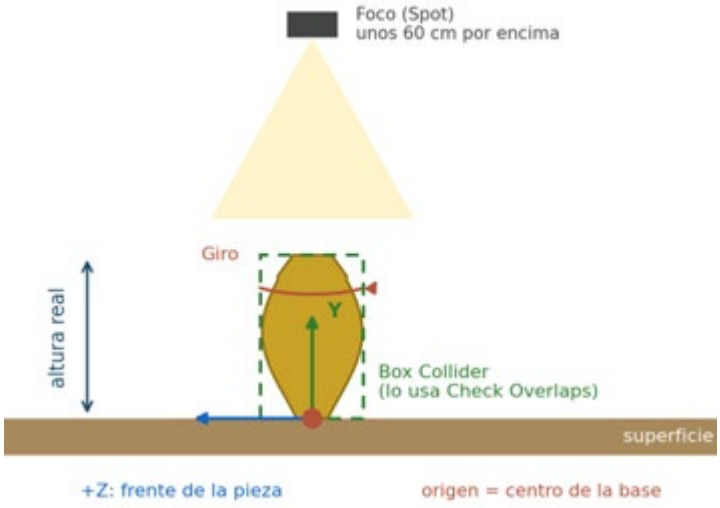
Crear un material sencillo con un color base libre y asignar al modelo.

e) Creación de BoxCollider

Con el GameObject *suzanne* seleccionado (el objeto hijo que tiene la malla de Suzanne), pulsar en el Inspector sobre el botón Add Component, y añadir un BoxCollider.

f) Colocar la SpotLight encima del objeto apuntando hacia abajo, y ajustar sus propiedades

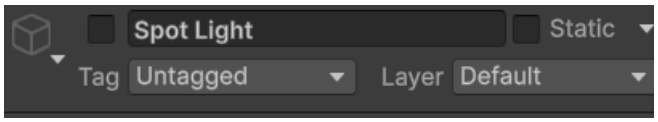
Se coloca unos 60 cm por encima, y mirando hacia abajo, como se muestra en el siguiente esquema.



Sus propiedades quedarían así.

- Posición y= 90 cm (NOTA: los valores de Unity son en m).
- Rotación = (90,0,0).
- General, Mode = Realtime.
- Shape, Outer Spot Angle = 40.
- Emission, Intensity = 3
- Emission, Range = 3
- Emission, Color = (255,235, 200) [Color cálido]
- Shadow Type = No Shadows

Por el momento, dejar la luz desactivada.



g) Creación de un script vacío derivado de Monobehaviour

Crear un script en la carpeta Assets – Scripts y vincularlo al objeto padre de la jerarquía (*suzanne_prefab*). El script lo dejamos vacío por el momento, pero así está preparado para luego poderle añadir comportamiento al prefab.

h) Creación del prefab

Arrastrar el objeto padre de la jerarquía a la carpeta prefabs. Una vez creado el prefab, se puede borrar la copia que hay en la escena.

3- Configuración para realidad mixta

Se puede crear una escena nueva en un proyecto ya configurado para XR, o configurar un nuevo proyecto para XR importando la librería Meta All-in-One SDK.

En la escena en la que se está trabajando, realizar los siguientes pasos:

- En primer lugar, si no se había hecho ya, eliminar la Main Camera del entorno y añadir el building block “Camera Rig”.

- Añadir el building block “Controller Tracking” para mostrar los controladores.
- Añadir el building block “Passthrough”.
- Añadir el building block “Effect Mesh”.

PRUEBA 1: Activar la opción “Set Play Mode to use MetaXRSimulator”. Ejecutar MetaXRSimulator para tenerlo ya abierto. Volver a Unity y pulsar play. Comprobar en el simulador que en la habitación de ejemplo “Bedroom” que lleva incluida se muestran superpuestas las mallas que indican la ubicación de los distintos objetos.

4- Mostrar el prefab creado haciendo uso del building block Find Spawn Positions.

Antes de seguir, es recomendable desactivar la representación gráfica del Effect Mesh. Para ello, activar la opción Hide Mesh en las propiedades del componente Effect Mesh.

También en las propiedades del Effect Mesh, se debe activar la opción Colliders, para que en siguientes apartados se pueda detectar si el objeto que se crea colisiona con algún objeto ya existente en el mundo real.

A continuación, ya se puede añadir el building block “Find Spawn Positions”.

En este building block, es importante configurar las siguientes opciones:

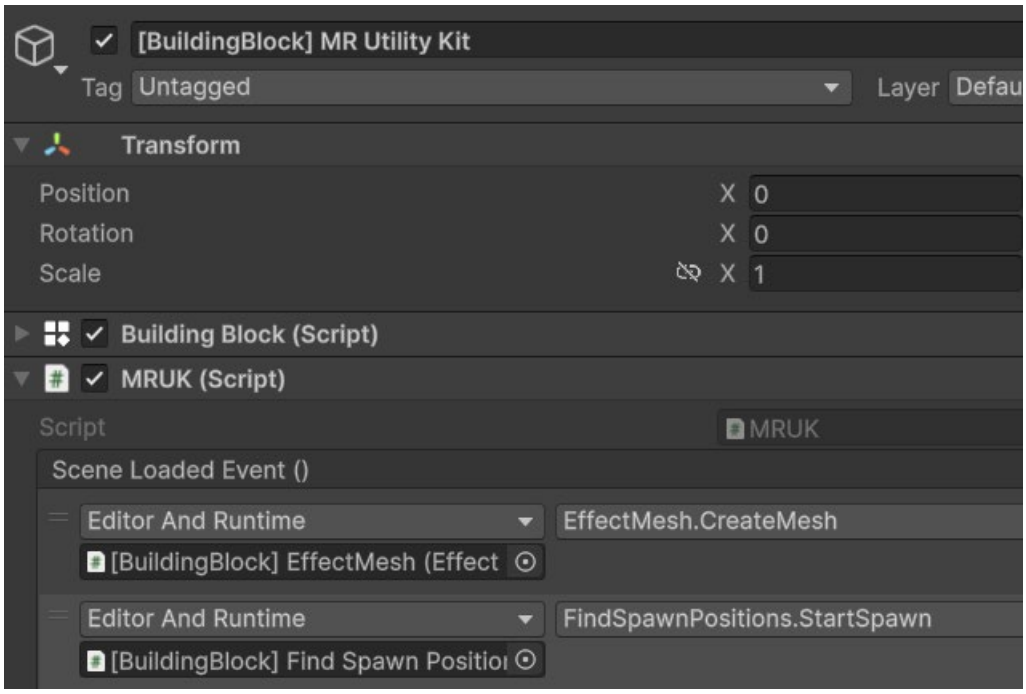
- Spawn On Start: Current Room Only.
- Spawn Object → Arrastrar el prefab Suzanne_prefab
- Spawn Amount: 1
- Spawn Locations: On Top Of Surfaces.
- Labels: TABLE
- Check Overlaps: marcado.
- Añadir el building block “Find Spawn Positions”. Marcar la opción de Check Overlaps.

PRUEBA 2: Comprobar en el simulador que se muestra la habitación de ejemplo y se ve el prefab sobre la mesa

5- Ordenar la ejecución de eventos.

Es importante que Effect Mesh cree sus colisionadores **antes** de que Spawn Location busque sitio para el objeto. Si no, Check Overlaps no verá paredes ni muebles y podría pasar que situara el objeto en una posición incorrecta, solapada con algún objeto del mundo real. Para conseguir que vayan en el orden adecuado, se hace esta configuración:

- En Find Spawn Positions, poner **Spawn On Start = None.**
- En Effect Mesh, poner **Spawn On Start = None**.
- Llamar a los métodos en el orden correcto desde el **Scene Loaded Event** de MRUK. Primero: Create Mesh. Segundo: StartSpawn. La forma correcta en la que deben quedar se ve en la siguiente figura.



PRUEBA 3: Comprobar en el simulador que se muestra la habitación de ejemplo y se ve el prefab sobre la mesa

6- Orientar el prefab por código para que mire inicialmente a la posición de la cámara

Para ello, modificar la función Start del script del prefab para que:

- Acceda a la posición de la Main Camera (que en el proyecto es CenterEyeAnchor)
- Calcule el vector dirección (que va de la posición del objeto a la posición inicial de la Main Camera).
- Compruebe si la magnitud del vector es mayor que 0, y en ese caso, cambie la rotación del objeto para que la dirección de mirada del objeto coincida con el vector dirección.

El código necesario se muestra a continuación.

```
cabeza = Camera.main.transform;


Vector3 direccion = cabeza.position - transform.position;
direccion.y = 0;
if (direccion.sqrMagnitude > 0.0001f){
   transform.rotation = Quaternion.LookRotation(direccion);
}
```

PRUEBA 4: Comprobar en el simulador que se muestra la habitación de ejemplo y se ve el prefab sobre la mesa mirando a la dirección inicial donde ha aparecido el usuario

7- Añadir el código para que por la pulsación del botón A del controlador izquierdo la pieza empiece a girar y se ilumine

Completar el script anterior para que cuando se pulse el botón A del controlador izquierdo, la pieza empiece a girar y se ilumine.

Estas son las variables necesarias que hay que definir en el script:

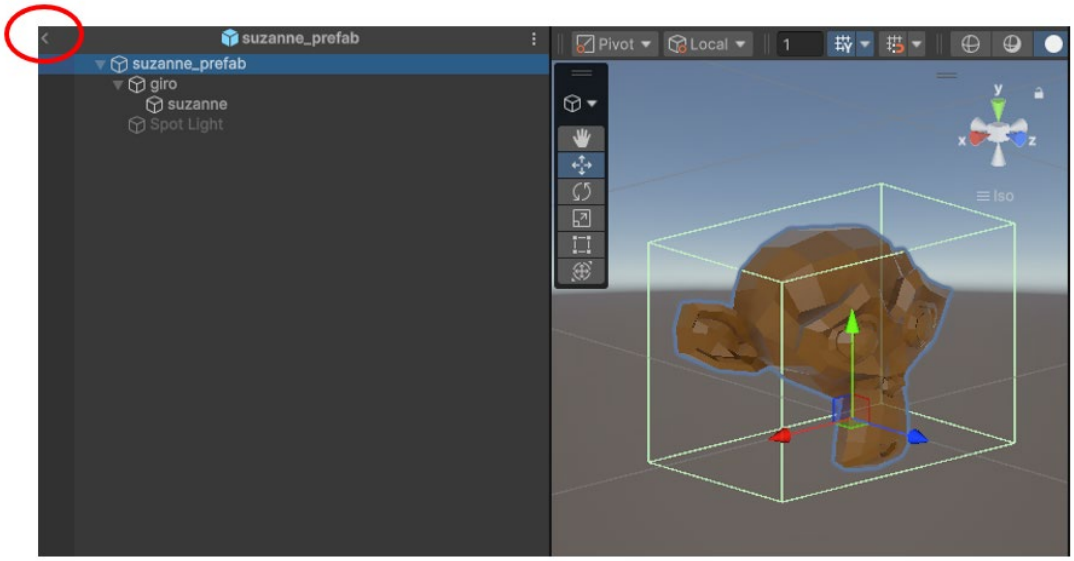
```
private bool girar;

private float velocidadGiro = 45.0f;

[SerializeField] private GameObject foco;

[SerializeField] private GameObject giro;
```

Hay que arrastrar los objetos hijos correspondientes del prefab sobre los campos de las variables que se verán en el editor (foco y giro). Esto se debe hacer desde el editor de prefabs. Para abrir el editor del prefab, es necesario hacer doble clic sobre el asset del prefab. Una vez asignadas las variables, se puede volver a la escena pulsando en la flecha que apunta hacia la izquierda.



El nuevo script debe:

- Crear un método Activar que haga que girar = true y active el objeto del foco de luz (haciendo uso de SetActive).
- Crear un método Desactivar que haga que girar=false y desactive el objeto del foco de luz.
- En el Update, comprobar si se ha pulsado el botón A del controlador derecho. Si se ha pulsado, ejecutar Activar o Desactivar según el estado actual del objeto.
- En el Update, una vez comprobado lo anterior, si la variable girar = true, aplicar un transform.Rotate al objeto *giro.*

PRUEBA 5: Comprobar en el simulador que se muestra la habitación de ejemplo y se ve el prefab sobre la mesa mirando a la dirección inicial donde ha aparecido el usuario. Se activa y desactiva con la pulsación del botón.

8- Entrega

La entrega se debe abordar y entregar en equipo, en base a los grupos creados en PoliformaT para las prácticas. Solo debe entregar uno de los miembros del equipo en la tarea creada en PoliformaT. Hay que entregar dos ficheros:

a) Vídeo

Una vez creado el proyecto de Unity siguiendo los pasos indicados en apartados anteriores, es necesario generar un vídeo mostrando aspectos seleccionados del mismo en el editor de Unity. El vídeo se puede generar haciendo uso de la herramienta de captura integrada en Windows (Windows + G). Debe mostrar paso a paso los siguientes aspectos del proyecto:

- La ventana de jerarquía, donde se vean los distintos GameObjects utilizados.
- La estructura de la carpeta Assets. Se deben mostrar las distintas carpetas generadas para organizar los Assets utilizados. Entrar expresamente a la carpeta prefabs, y hacer doble clic en el prefab para mostrar la estructura del mismo.
- Pulsar el botón play y mostrar el funcionamiento de la aplicación definitiva en el Meta XR Simulator. Se debe poder ver en el vídeo cómo el objeto se activa y se desactiva.

b) Proyecto de Unity en zip

Se trata de un fichero zip que incluya las siguientes carpetas del proyecto de Unity:

- **Assets.**
- **Packages.**
- **Project Settings.**